Deep Network Development

Autumn 2026, Practice #4

Viktor Varga
Department of Artificial Intelligence, ELTE IK

Last week - Introduction to NumPy



Last week - Introduction to NumPy

The classic approach

```
my_list = [1, 2, 3, -42, 99]

for i in range(len(my_list)):
    my_list[i] += 1

print(my_list)

>>> [2, 3, 4, -41, 100]
```

Scalar operation: scalar + scalar

Array programming

```
import numpy as np

my_array = np.array([1, 2, 3, -42, 99])

my_array += 1

print(my_array)

>>> [  2   3   4 -41 100]
```

Vectorized operation: array + scalar

Image source: https://www.geeksforgeeks.org/python/python-lists-vs-numpy-arrays/

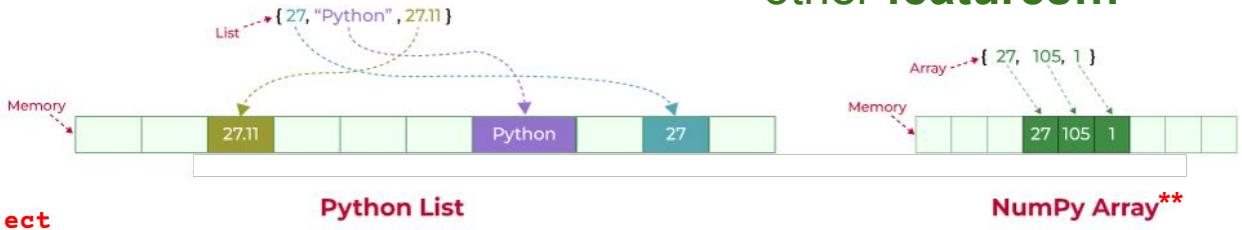
Last week - Python lists vs. NumPy arrays

Python lists

- Can have **mixed type** elements
  → **Not efficient** (references)
- Has some free space at the end
  → **list.append is O(1)**

NumPy arrays

- Elements have a **fixed datatype***
  → **Efficient** (native storage)
- Completely fills reserved memory**
  → **ndarray.append is O(n)**
- Can be **multi-dimensional**
- Much more **powerful** indexing and other **features…**



* Except for `dtype=object`
** Only true for base arrays, not necessarily for views.

Last week - The ndarray type

The ndarray type – Main attributes:

```
a = np.array([0., 10., 20., 30., 40.], dtype=np.float32)
```

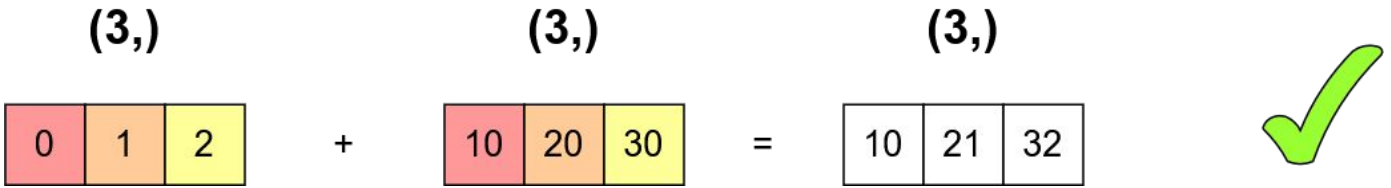
- `ndarray.size` → `5`
- `ndarray.ndim` → `1`
- `ndarray.shape` → `(5,)`
- `ndarray.dtype` → `float32`
- `ndarray.itemsize` → `4`
- `ndarray.strides`
- `ndarray.base`

Last week - NumPy

Element-wise operations

```
a = np.array([0, 1, 2], dtype=np.int64)          # a.shape == (3,)
b = np.array([10, 20, 30], dtype=np.int64)       # b.shape == (3,)

print((a+b).shape)                               # --> (3,)
print(a+b)                                       # --> [10 21 32]
```

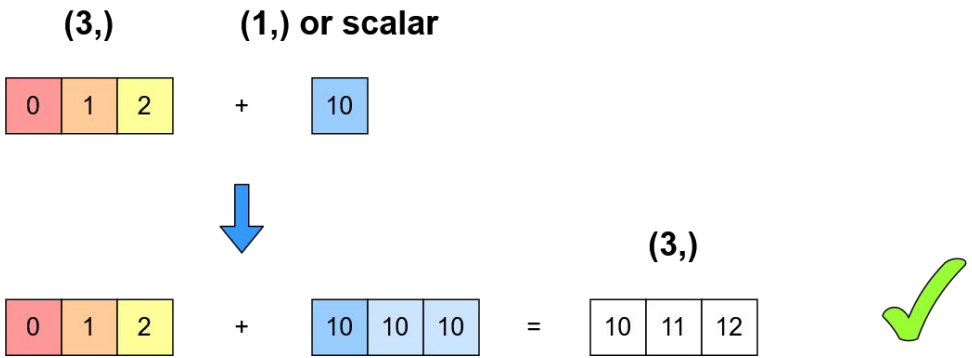


Last week - NumPy

Element-wise operations with broadcasting (1D)

```
a = np.array([0, 1, 2], dtype=np.int64)      # a.shape == (3,)

print((a + 10).shape)                        # --> (3,)
print(a + 10)                                # --> [10 11 12]
```

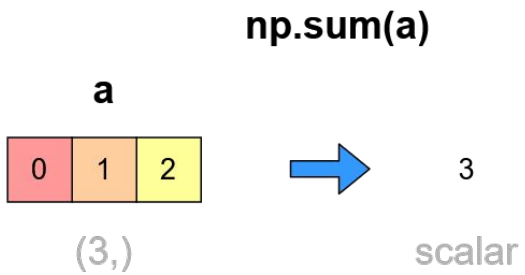


Last week - NumPy

Collapse operations (1D)

```
a = np.array([0, 1, 2], dtype=np.int64)      # a.shape == (3,)

print(np.sum(a))                             # --> 3
```



Last week - NumPy

Indexing (1D)

1. `my_array[0]` — **Individual integer index**
2. `my_array[1:4:2]` — **Slicing**
3. `my_array[[-2, 0, 1, 0]]` — **Indexing with an integer list / array**
4. `my_array[[True, True, False, True, False]]`
   `my_array[my_array % 3 == 0]` — **Indexing with a bool array (mask)**

Last week - NumPy

Indexing (1D)

1. `my_array[0]`

   Individual integer index
   → Scalar

2. `my_array[1:4:2]`

   Slicing
   → A (view) **array***

3. `my_array[[-2, 0, 1, 0]]`

   Indexing with an integer list / array
   → A new array

4. `my_array[[True, True, False, True, False]]`
   `my_array[my_array % 3 == 0]`

   Indexing with a bool array (mask)
   → A new array

* In most cases

Last week - NumPy

Indexing for overwriting elements of an array (1D)

1

```
my_array[0] = 42
```

Individual integer index

2

```
my_array[1:4:2] = 42, 99
```

Slicing

Indexing with an integer list / array

3

```
my_array[[-2, 0, 1, 0]] = 42, 99, 11, 75
```

4

```
my_array[[True, True, False, True, False]] = 42, 99, 11
my_array[my_array % 30 == 0] = 42, 99
```

Indexing with a bool array (mask)

Last week - NumPy

Indexing for overwriting elements of an array (1D)

1

```
my_array[0] = my_int
```

Individual integer index

2

```
my_array[1:4:2] = my_tuple
```

Slicing

Indexing with an integer list / array

3

```
my_array[[-2, 0, 1, 0]] = my_list
```

4

```
my_array[[True, True, False, True, False]] = my_other_array
my_array[my_array % 30 == 0] = my_other_array
```

Indexing with a bool array (mask)

Last week - NumPy

Broadcasting when overwriting elements of an array (1D)

N elements = N values

```
my_array[[1, 3, 0]] = 42, 99, 11
```



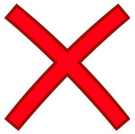
N elements = 1 value

```
my_array[[1, 3, 0]] = 42
```



N elements = M values (N ≠ M ≠ 1)

```
my_array[[1, 3, 0]] = 42, 99
```



Last week - NumPy

Finding element(s) based on a mask / condition (1D)

m.shape: (3,)
m.dtype == np.bool_

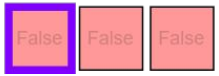


np.where(m) --> **(1 2 ,)**



np.argmax(m) --> **1**

More efficient



np.argmax(m) --> **0**

NumPy - Multidimensional arrays

The motivation for multidimensional arrays

- Storing a tabular dataset

```
(n_examples, n_variables)
```

NumPy - Multidimensional arrays

The motivation for multidimensional arrays

- Storing a tabular dataset
- Storing a grayscale image

```
(n_examples, n_variables)
(height, width)
```

NumPy - Multidimensional arrays

The motivation for multidimensional arrays

- Storing a tabular dataset
- Storing a grayscale image
- Storing a color image

```
(n_examples, n_variables)
(height, width)
(height, width, 3)
```

NumPy - Multidimensional arrays

The motivation for multidimensional arrays

- Storing a tabular dataset
- Storing a grayscale image
- Storing a color image
- Storing a color video

```
(n_examples, n_variables)
(height, width)
(height, width, 3)
(n_frames, height, width, 3)
```

NumPy - Multidimensional arrays

The motivation for multidimensional arrays

- Storing a tabular dataset
- Storing a grayscale image
- Storing a color image
- Storing a color video
- Storing the output of a FC neuron layer

```
(n_examples, n_variables)
(height, width)
(height, width, 3)
(n_frames, height, width, 3)
(batch_size, n_neurons)
```

NumPy - Multidimensional arrays

The motivation for multidimensional arrays

- Storing a tabular dataset
- Storing a grayscale image
- Storing a color image
- Storing a color video
- Storing the output of a FC neuron layer
- Storing the output of a convolutional layer
- …

```
(n_examples, n_variables)
(height, width)
(height, width, 3)
(n_frames, height, width, 3)
(batch_size, n_neurons)
(batch_size, n_channels, h, w)
```

NumPy - Multidimensional arrays

The motivation for multidimensional arrays

- Storing a tabular dataset
- Storing a grayscale image
- Storing a color image
- Storing a color video
- Storing the output of a FC neuron layer
- Storing the output of a convolutional layer
- …

```
(n_examples, n_variables)
(height, width)
(height, width, 3)
(n_frames, height, width, 3)
(batch_size, n_neurons)
(batch_size, n_channels, h, w)
```

However, we often just need **temporary multidimensional arrays** to be able **to perform a certain** vectorized **operation…**

NumPy - Multidimensional array example

This week's example:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

NumPy - Multidimensional array example

The ndarray type – Main attributes:

```
self.board = np.zeros((9, 9), dtype=np.uint8)
```

- `ndarray.size` → `81`
- `ndarray.ndim` → `2`
- `ndarray.shape` → `(9, 9)`
- `ndarray.dtype` → `uint8`
- `ndarray.itemsize` → `1`
- `ndarray.strides`
- `ndarray.base`

NumPy - Array creation methods (Multidimensional)

```
self.board = np.zeros((9, 9), dtype=np.uint8)                                  # (9, 9)
zeroelement_array = np.zeros((2, 0, 3), dtype=np.float32)                      # (2, 0, 3)


array_3x2 = np.array([[0, 1], [2, 3], [4, 5]], dtype=np.int64)                   # (3, 2)
array_2x3 = np.array([[False, True, False], [True, True, False]], dtype=np.bool_)   # (2, 3)

array_2x2x3 = np.array([[[0, 1], [2, 3]], [[4, 5], [6, 7]]], dtype=np.int64)     # (2, 2, 2)

array_1x2x1x1 = np.array( [[ [[42]], [[43]] ]] , dtype=np.int64)                 # (1, 2, 1, 1)



array_fail = np.array([[0], [1, 2], [3]] , dtype=np.int64)
# --> ValueError: ... The requested array has an inhomogeneous shape ...
```

NumPy - Array creation methods (Multidimensional)

```
self.board = np.zeros((9, 9), dtype=np.uint8)                                  # (9, 9)
zeroelement_array = np.zeros((2, 0, 3), dtype=np.float32)                      # (2, 0, 3)
```

```
array_3x2 = np.array([[0, 1], [2, 3], [4, 5]], dtype=np.int64)                   # (3, 2)
array_2x3 = np.array([[False, True, False], [True, True, False]], dtype=np.bool_)    # (2, 3)

array_2x2x3 = np.array([[[0, 1], [2, 3]], [[4, 5], [6, 7]]], dtype=np.int64)      # (2, 2, 2)

array_1x2x1x1 = np.array( [[ [[42]], [[43]] ]] , dtype=np.int64)                  # (1, 2, 1, 1)
```

Creating an array from a **nested list structure**:
The array axes go from the outer levels to the inner levels.

```
# --> ValueError: ... The requested array has an inhomogeneous shape ...
```

NumPy - Array creation methods (Multidimensional)

```
self.board = np.zeros((9, 9), dtype=np.uint8)                                  # (9, 9)
zeroelement_array = np.zeros((2, 0, 3), dtype=np.float32)                      # (2, 0, 3)


array_3x2 = np.array([[0, 1], [2, 3], [4, 5]], dtype=np.int64)                   # (3, 2)
array_2x3 = np.array([[False, True, False], [True, True, False]], dtype=np.bool_)   # (2, 3)

array_2x2x3 = np.array([[[0, 1], [2, 3]], [[4, 5], [6, 7]]], dtype=np.int64)     # (2, 2, 2)

array_1x2x1x1 = np.array( [[ [[42]], [[43]] ]] , dtype=np.int64)                 # (1, 2, 1, 1)
```

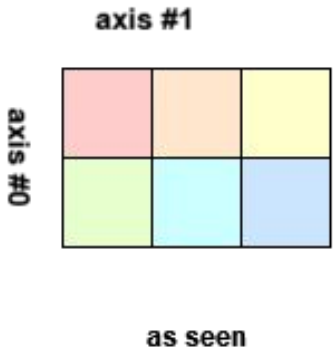
Failure in case of non-uniform lengths in a level…

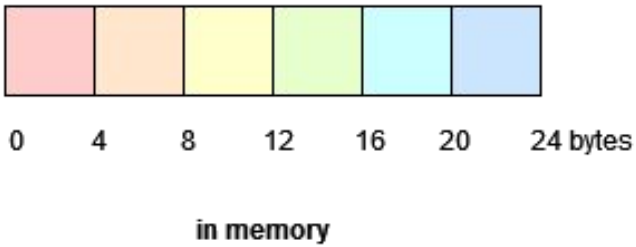
```
array_fail = np.array([[0], [1, 2], [3]] , dtype=np.int64)
# --> ValueError: ... The requested array has an inhomogeneous shape ...
```

NumPy - Multidimensional arrays

```
a = np.zeros((2, 3), dtype=np.int64)
print(type(a))
print(a)
```

```
# a.shape == (2, 3)
# <class 'numpy.ndarray'>
#   [[0 0 0]
#    [0 0 0]]
```

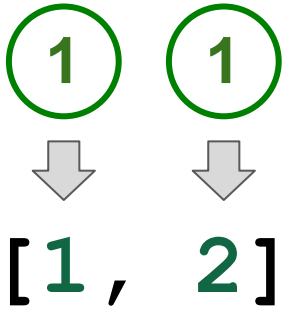




NumPy - Array indexing



Indexing an individual element of the nD array.
Returns a scalar.



Indexing of an individual element from an array:

- **All axes** of an array are indexed using an **individual index.**
- A single element is returned as a **scalar.**

NumPy - Array indexing



Indexing an individual element of the nD array.
Returns a scalar.

```
a = np.array([[0, 1, 2], [10, 11, 12]], dtype=np.int64)   # a.shape == (2, 3)
print(type(a))             # <class 'numpy.ndarray'>
print(a)                   #   [[ 0  1  2]
                           #    [10 11 12]]

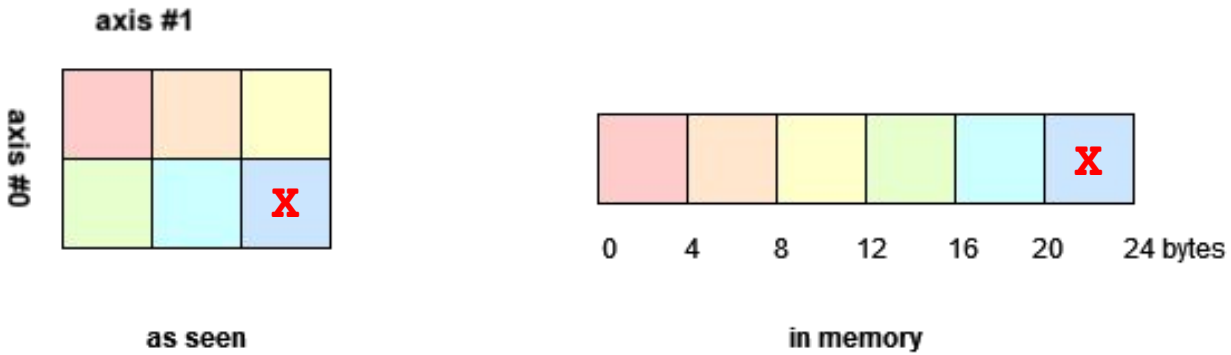
print( a[0, 2] )           # 2
print( type(a[0, 2]) )     # <class 'numpy.int64'>
print( a[-1, 0] )          # 10
print( a[-1, -2] )         # 11

print( a[2, 0] )           # IndexError: index 2 is out of bounds for axis 0 with size 2
```

NumPy - Multidimensional arrays

```
a = np.zeros((2, 3), dtype=np.int64)
print(type(a))
print(a)
```

```
# a.shape == (2, 3)
# <class 'numpy.ndarray'>
#   [[0 0 0]
#    [0 0 0]]
```



E.g., the memory address of **`a[1, 2]`**, will be the address of **`a[0, 0]`** plus **`(1*3 + 2*1)*a.itemsize`** bytes.

NumPy - Array indexing



Indexing an individual element of the nD array.
Returns a scalar.

```
self.board[row_idx, col_idx]
```

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

NumPy - Array indexing for **writing**

```
a = np.array([[0, 1, 2], [10, 11, 12]], dtype=np.int64)    # a.shape == (2, 3)
```




```
a[-1, -2] = 99
print(a)                    #    [[ 0   1   2]
                            #     [10 99 12]]
```

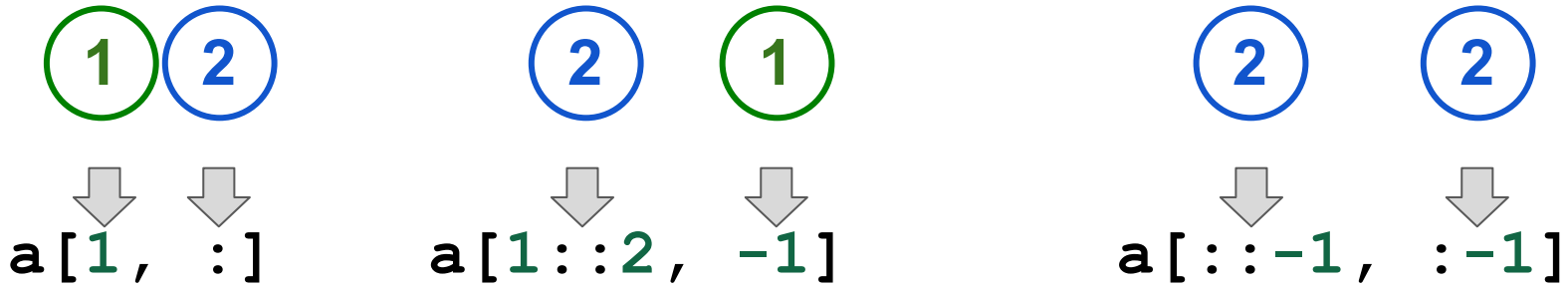
Overwriting a single element by using individual element indexing

Reminder: The right hand-side value is converted to the datatype of the array.

NumPy - Array indexing

B

Basic indexing.
Returns a view array over the original array.*



Basic indexing:

- Can only use a **combination of individual indexing and slicing.**
- **At least one slicing** is needed.**
- An array is returned which is a view over the original array.*
- Each **axis with individual index will be collapsed** in the result.
- Each **axis with slicing** will be kept** in the result (even if length is zero).

* **Always true for base arrays. Often true for view arrays, but in certain cases not.**
** **Can be implicit.**

NumPy - Array indexing

B

Basic indexing.
Returns a view array over the original array.*

```
a = np.array([[0, 1, 2], [10, 11, 12]], dtype=np.int64)    # a.shape == (2, 3)
print(type(a))             # <class 'numpy.ndarray'>
print(a)                   #    [[ 0  1  2]
                           #     [10 11 12]]
```

```
print( a[0, :] )           # [0 1 2]
print( a[0, :].shape )     # (3,)
print( type(a[0, :]) )     # <class 'numpy.ndarray'>
```

1 2

```
print( a[:, -2] )           # [1 11]
print( a[:, -2].shape )     # (2,)
```

2 1

NumPy - Array indexing

B

Basic indexing.
Returns a view array over the original array.*

```
a = np.array([[0, 1, 2], [10, 11, 12]], dtype=np.int64)    # a.shape == (2, 3)
print(type(a))             # <class 'numpy.ndarray'>
print(a)                   #    [[ 0  1  2]
                           #     [10 11 12]]
```

```
print( a[0, :] )           # [0 1 2]
print( a[0, :].shape )     # (3,)
print( type(a[0, :]) )     # <class 'numpy.ndarray'>
```

1 2

```
print( a[:, -2] )           # [1 11]
print( a[:, -2].shape )     # (2,)
```

2 1

NumPy - Array indexing



Basic indexing.
Returns a view array over the original array.*

```
a = np.array([[0, 1, 2], [10, 11, 12]], dtype=np.int64)    # a.shape == (2, 3)
print(type(a))             # <class 'numpy.ndarray'>
print(a)                   #    [[ 0  1  2]
                           #     [10 11 12]]
```

```
print( a[0, ::-1] )          # [2 1 0]
print( a[0, ::-1].shape )    # (3,)
```




```
print( a[1:2, -1] )          # [12]
print( a[1:2, -1].shape )    # (1,)
```

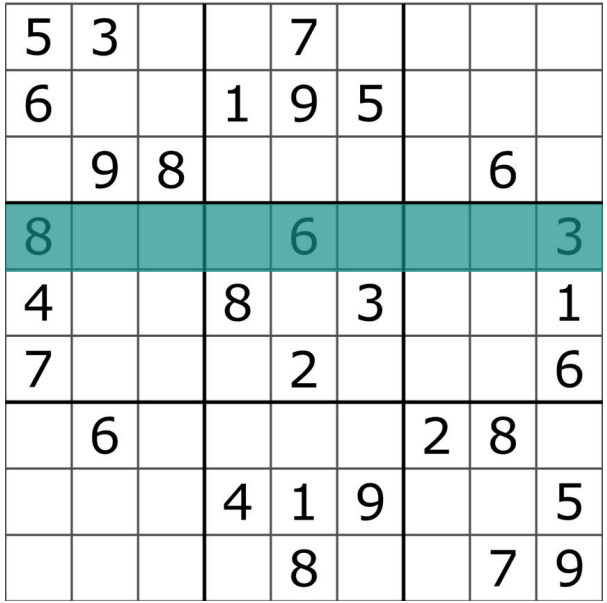


NumPy - Array indexing

B

Basic indexing.
Returns a view array over the original array.*

```
self.board[row_idx, :]
```



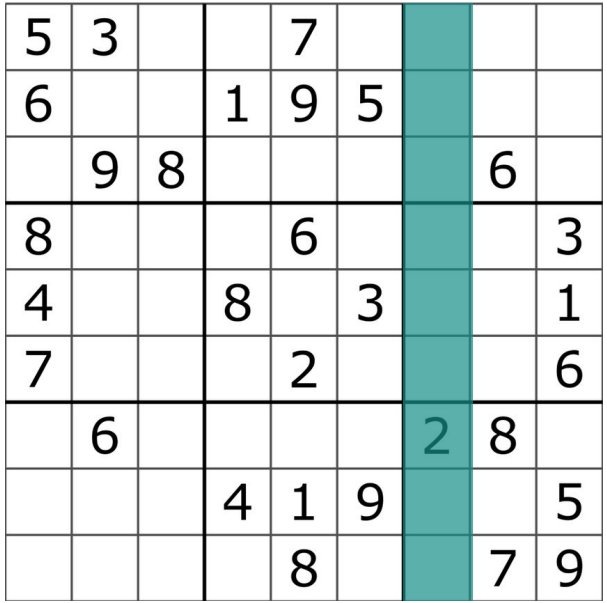
Result: 1D

NumPy - Array indexing

B

Basic indexing.
Returns a view array over the original array.*

```
self.board[:, col_idx]
```



Result: 1D

NumPy - Array indexing

B

Basic indexing.
Returns a view array over the original array.*

```
a = np.array([[0, 1, 2], [10, 11, 12]], dtype=np.int64)    # a.shape == (2, 3)
print(type(a))             # <class 'numpy.ndarray'>
print(a)                   #    [[ 0  1  2]
                           #     [10 11 12]]
```

2 2

```
print( a[::-1, ::2] )          #    [[10 12]
                               #     [ 0  2]]
print( a[::-1, ::2].shape )    # (2, 2)
```

```
print( a[-2:, :] )             #    [[ 0  1  2]
                               #     [10 11 12]]
print( a[-2:, :].shape )       # (2, 3)
```

```
print( a[:, :].shape )     # (2, 3) - same content as a
```

NumPy - Array indexing

B

Basic indexing.
Returns a view array over the original array.*

```
self.board[ block_start_row_idx : block_start_row_idx+3 , \
            block_start_col_idx : block_start_col_idx+3 ]
```

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

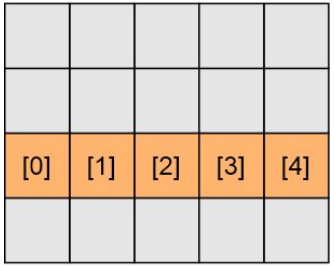
Result: 2D

NumPy - Array indexing

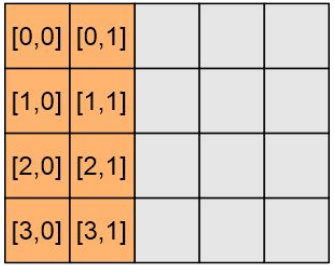
B

Basic indexing.
Returns a view array over the original array.*

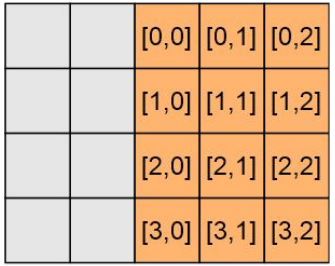
a[2, :]



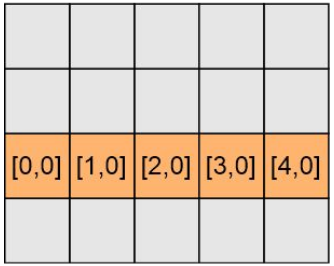
a[:, :2]



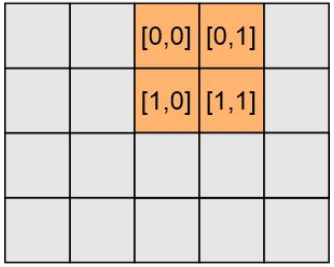
a[:, 2:]



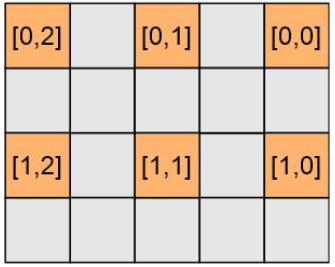
a[2:3, :]



a[:2, 2:-1]



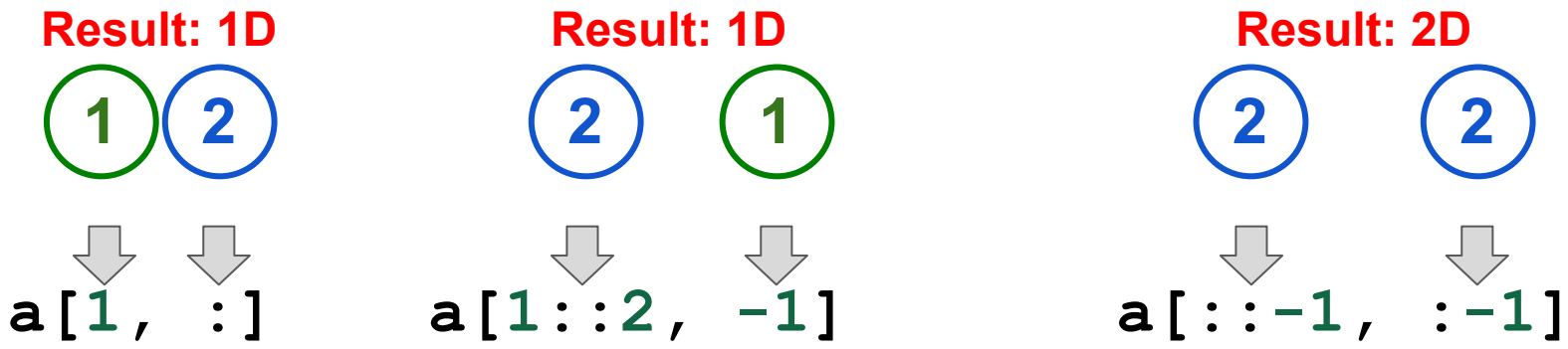
a[:3:2, -1::-2]



NumPy - Array indexing

B

Basic indexing.
Returns a view array over the original array.*



Basic indexing:

- Can only use a **combination of individual indexing and slicing.**
- **At least one slicing** is needed.**
- An array is returned which is a view over the original array.*
- Each **axis with individual index will be collapsed** in the result.
- Each **axis with slicing** will be kept** in the result (even if length is zero).

* **Always true for base arrays. Often true for view arrays, but in certain cases not.**
** **Can be implicit.**

NumPy - Array indexing



Basic indexing.
Returns a view array over the original array.*

Implicit slicing - Syntactic sugar

`a[0]` **is equivalent to** `a[0]` **or** `a[0, :]` **or** `a[0, :, :]` **or …**

`a[..., -2]` **is equivalent to** `a[-2]` **or** `a[:, -2]` **or** `a[:, :, -2]` **or …**

… depending on **`a.ndim`**

NumPy - Array indexing for **writing**

```
a = np.array([[0, 1, 2], [10, 11, 12]], dtype=np.int64)    # a.shape == (2, 3)
b = np.array([[42, 99], [77, 81]], dtype=np.int32)         # b.shape == (2, 2)
```

B

```
a[::-1, 1:] = b
print(a)                    #    [[ 0 77 81]
                                  [10 42 99]])
```

Selecting a (2, 2) area ⟷ Overwriting these with values from a (2, 2) array.

Same shape on both sides of the assignment.



NumPy - Array indexing for **writing**

```
a = np.array([[0, 1, 2], [10, 11, 12]], dtype=np.int64)    # a.shape == (2, 3)
```

B

```
a[::-1, 1:] = 99
print(a)                    #    [[ 0 99 99]
                                  [10 99 99]])
```

Selecting a (2, 2) area ⟷ Overwriting these with a single value.

(M, N) vs. (1,) or scalar shape on the two sides of the assignment

→ **BROADCASTING**
(Virtual repetitions)



NumPy - Array indexing for **writing**

```
a = np.array([[0, 1, 2], [10, 11, 12]], dtype=np.int64)    # a.shape == (2, 3)
```

B

```
a[::-1, 1:] = 42, 99, 77, 81
# ValueError: could not broadcast input array from shape (4,) into shape (2,2)
```

Selecting a (2, 2) area ⟷ Overwriting these with values arranged in a shape of (4,).

(M, N) vs. (K,) shape, N ≠ K ≠ 1 → BROADCASTING FAILS



Exact broadcasting rules: next week!

Reminder - Examples for element-wise operations

Numeric × Numeric → Boolean

```
==    (np.equal)
!=    (np.not_equal)
<     (np.less)
<=    (np.less_equal)
>     (np.greater)
>=    (np.greater_equal)
np.isclose

...
```

Boolean × Boolean → Boolean

```
==    (np.equal)
!=    (np.not_equal)
&     (np.logical_and)
|     (np.logical_or)
np.logical_xor
...
```

Boolean → Boolean

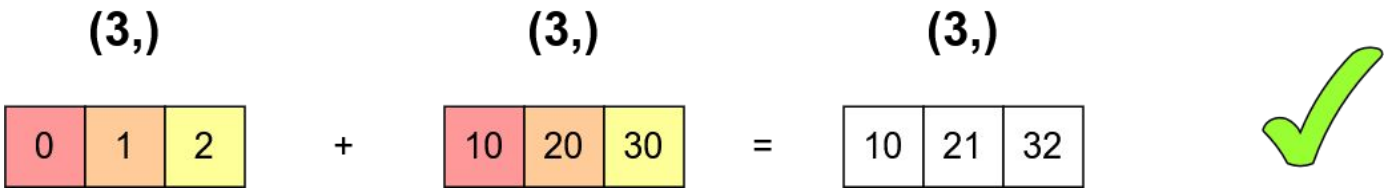
```
~     (np.logical_not)
```

Reminder - Element-wise operations

Element-wise operations can be applied on arrays with similar shape…

```
a = np.array([0, 1, 2], dtype=np.int64)          # a.shape == (3,)
b = np.array([10, 20, 30], dtype=np.int64)       # b.shape == (3,)

print((a+b).shape)                               # --> (3,)
print(a+b)                                       # --> [10 21 32]
```

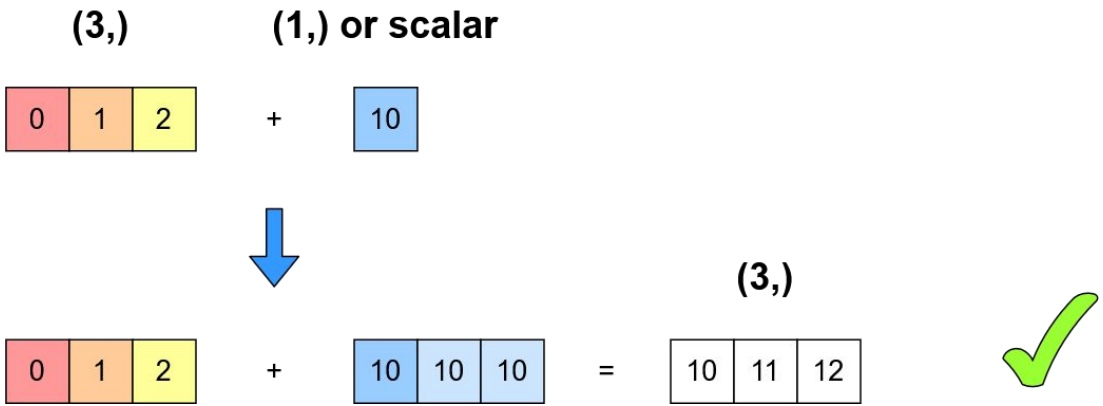


Reminder - Element-wise ops with broadcasting (1D)

… or **the shapes need to be compatible** in terms of **broadcasting.**

```
a = np.array([0, 1, 2], dtype=np.int64)      # a.shape == (3,)

print((a + 10).shape)                        # --> (3,)
print(a + 10)                                # --> [10 11 12]
```

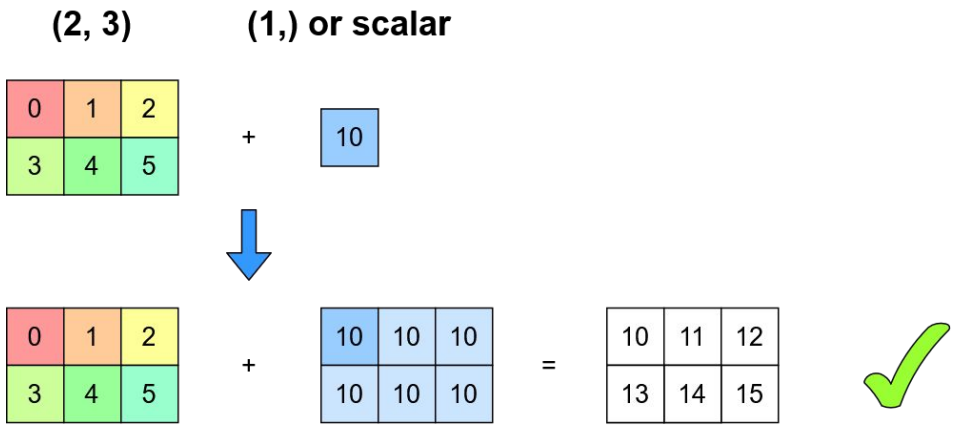


NumPy - Element-wise ops with broadcasting, **(m, n) vs. (1,)**

… or **the shapes need to be compatible** in terms of **broadcasting.**

```
a = np.array([[0, 1, 2], [3, 4, 5]], dtype=np.int64)    # a.shape == (2, 3)


print((a + 10).shape)                                   # --> (2, 3)
print(a + 10)                                           # --> [[10 11 12]
                                                        #      [13 14 15]]
```



Reminder - Examples for collapse operations (1D)

[Numeric] → Numeric

```
np.sum      (np.ndarray.sum)
np.prod     (np.ndarray.prod)
np.amax     (np.ndarray.max)
np.amin     (np.ndarray.min)
np.mean     (np.ndarray.mean)
np.var      (np.ndarray.var)
np.std      (np.ndarray.std)
np.median
np.argmax   (np.ndarray.argmax)
np.argmin   (np.ndarray.argmin)
np.count_nonzero
...
```

[Boolean] → Boolean

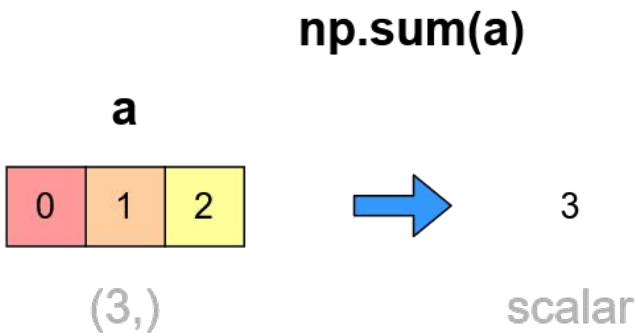
```
np.any    (np.ndarray.any)
np.all    (np.ndarray.all)
np.count_nonzero
...
```

Reminder - Collapse operations (1D)

(1D case) **Collapse operations** collapse an 1D array into a scalar.

```
a = np.array([0, 1, 2], dtype=np.int64)      # a.shape == (3,)

print(np.sum(a))                             # --> 3
```

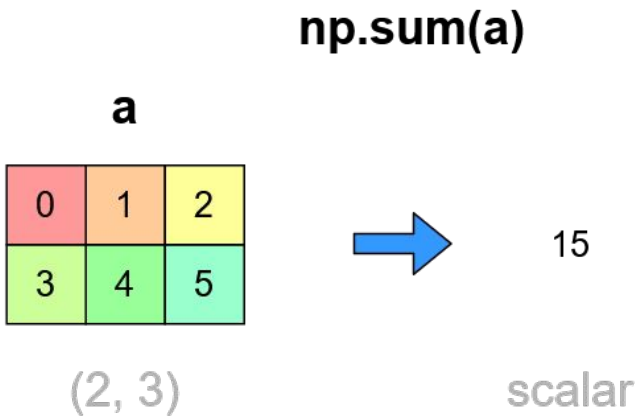


NumPy - Collapse operations (no axis parameter case)

Collapse operations can collapse a multi-dimensional array into a scalar.

```
a = np.array([[0, 1, 2], [3, 4, 5]], dtype=np.int64)    # a.shape == (2, 3)

print(np.sum(a))                                        # --> 15
```

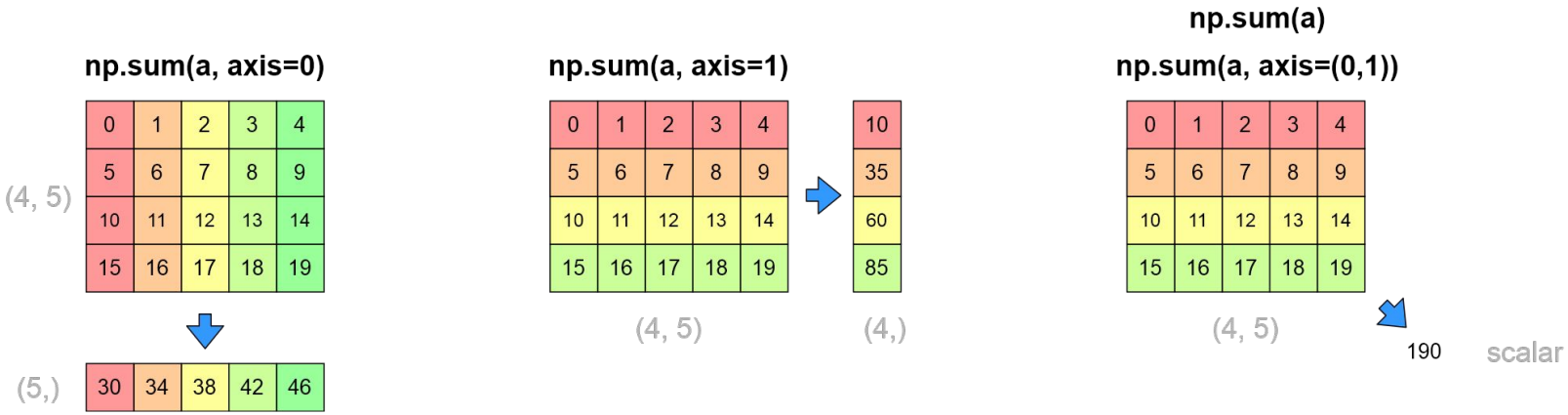


NumPy - Collapse operations (with an axis parameter)

Collapse operations can collapse only selected axes of an array.

```
a = np.arange(20, dtype=np.int64).reshape(4, 5)     # a.shape == (4, 5)

print(np.sum(a, axis=0))                            # --> [30 34 38 42 46],  a.shape == (5,)
print(np.sum(a, axis=1))                            # --> [10 35 60 85],     a.shape == (4,)
print(np.sum(a, axis=(0,1)))                        # --> 190
print(np.sum(a))                                    # --> 190
```

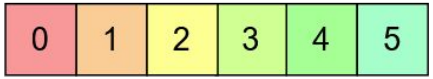


NumPy - Reshaping arrays

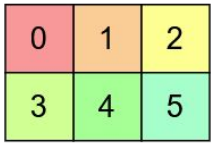
```
a6 = np.arange(6, dtype=np.int64)        # a6.shape == (6,)

a2x3 = a6.reshape((2, 3))                # a2x3.shape == (2, 3)
a3x2 = a2x3.reshape((3, 2))              # a3x2.shape == (3, 2)

print(a6)                                #    [0 1 2 3 4 5]
```




```
print(a2x3)                              #    [[0 1 2]
                                         #     [3 4 5]]
```




```
print(a3x2)                              #    [[0 1]
                                         #     [2 3]
                                         #     [4 5]]
```



NumPy - Reshaping arrays

```
a6 = np.arange(6, dtype=np.int64)        # a6.shape == (6,)

a2x3 = a6.reshape((2, 3))                # a2x3.shape == (2, 3)
a3x2 = a2x3.reshape((3, 2))              # a3x2.shape == (3, 2)

a3x2[::2, 1] = 99

print(a3x2)                              #    [[0 99]
                                         #     [2  3]
                                         #     [4 99]]

print(a6)                                #    [0 99 2 3 4 99]

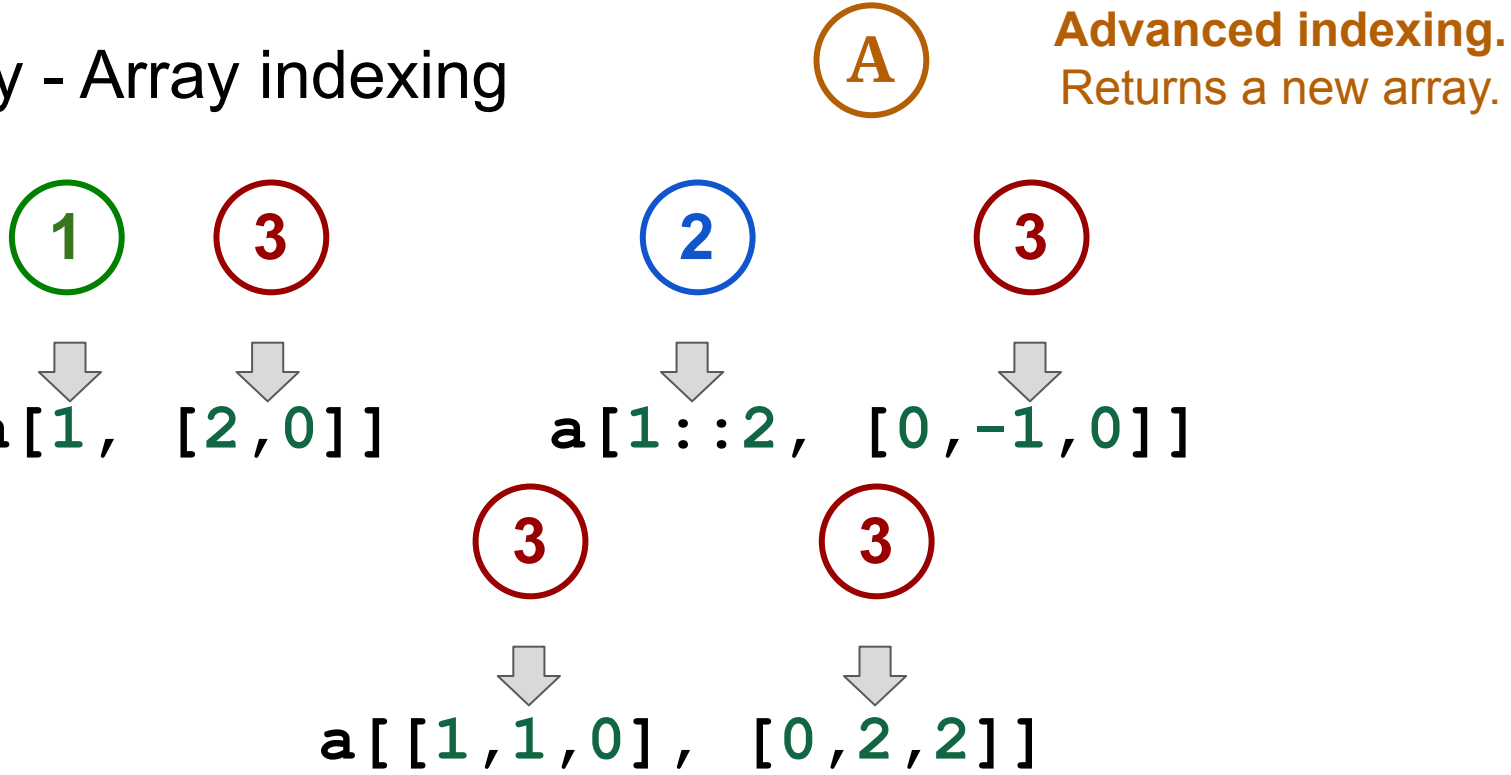
print(a2x3)                              #    [[0 99 2]
                                         #     [3 4 99]]
```

Views of the original array.* Memory is shared.

Writing into any of the views or into the original array will **affect all** of them.

* Always true for base arrays. Often true for view arrays, but in certain cases not.

NumPy - Array indexing



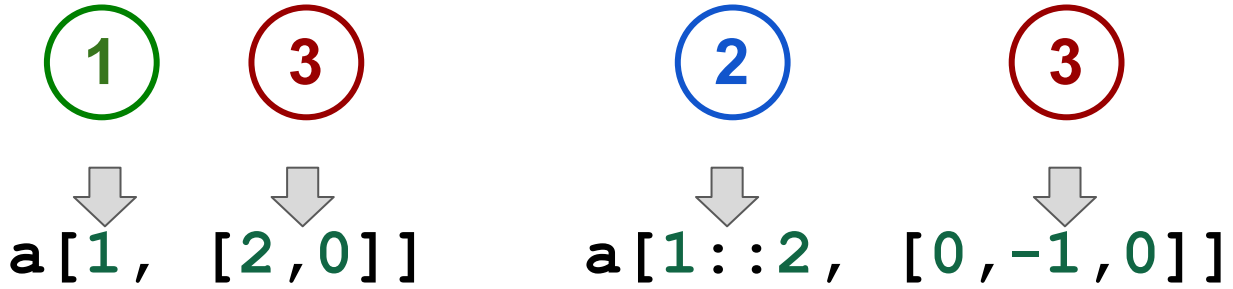
Advanced indexing:

- Will happen if either integer sequence or mask indexing is used at least once.
- A **new array** is returned with the selected elements copied into it.

NumPy - Array indexing



Advanced indexing.
Returns a new array.



The simplest case: Two integer arrays of the same shape for indexing.

Will select: a[1,0], a[1,2], a[0,2]

Advanced indexing:

- Will happen if either integer sequence or mask indexing is used at least once.
- A **new array** is returned with the selected elements copied into it.

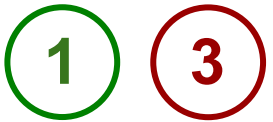
NumPy - Array indexing



Advanced indexing.
Returns a new array.

```
a = np.array([[0, 1, 2], [10, 11, 12]], dtype=np.int64)    # a.shape == (2, 3)
print(a)                    #    [[ 0  1  2]
                            #     [10 11 12]]
```

```
print( a[1, [2, 0]] )           # [12 10]
print( a[1, [2, 0]].shape )     # (2,)
print( type(a[0, :]) )     # <class 'numpy.ndarray'>
```




```
print( a[1::2, [0,-1,0]] )           # [[10 12 10]]
print( a[1::2, [0,-1,0]].shape )     # (1, 3)
```




```
print( a[[1,1,0], [0,2,2]] )           # [10 12  2]
print( a[[1,1,0], [0,2,2]].shape )     # (3,)
```

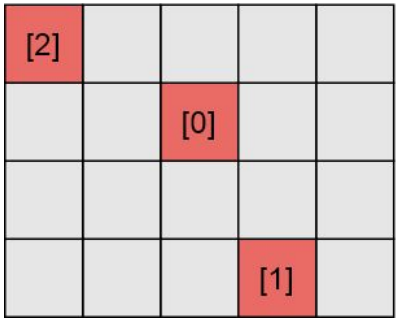


NumPy - Array indexing

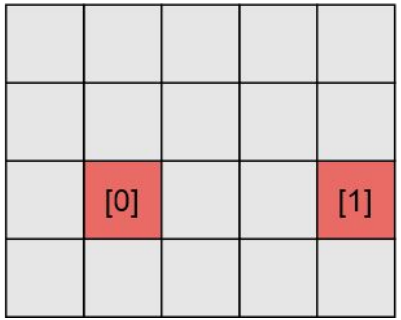


Advanced indexing.
Returns a new array.

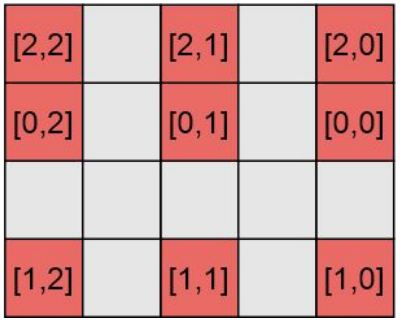
a[[1,3,0], [2,-2,0]]



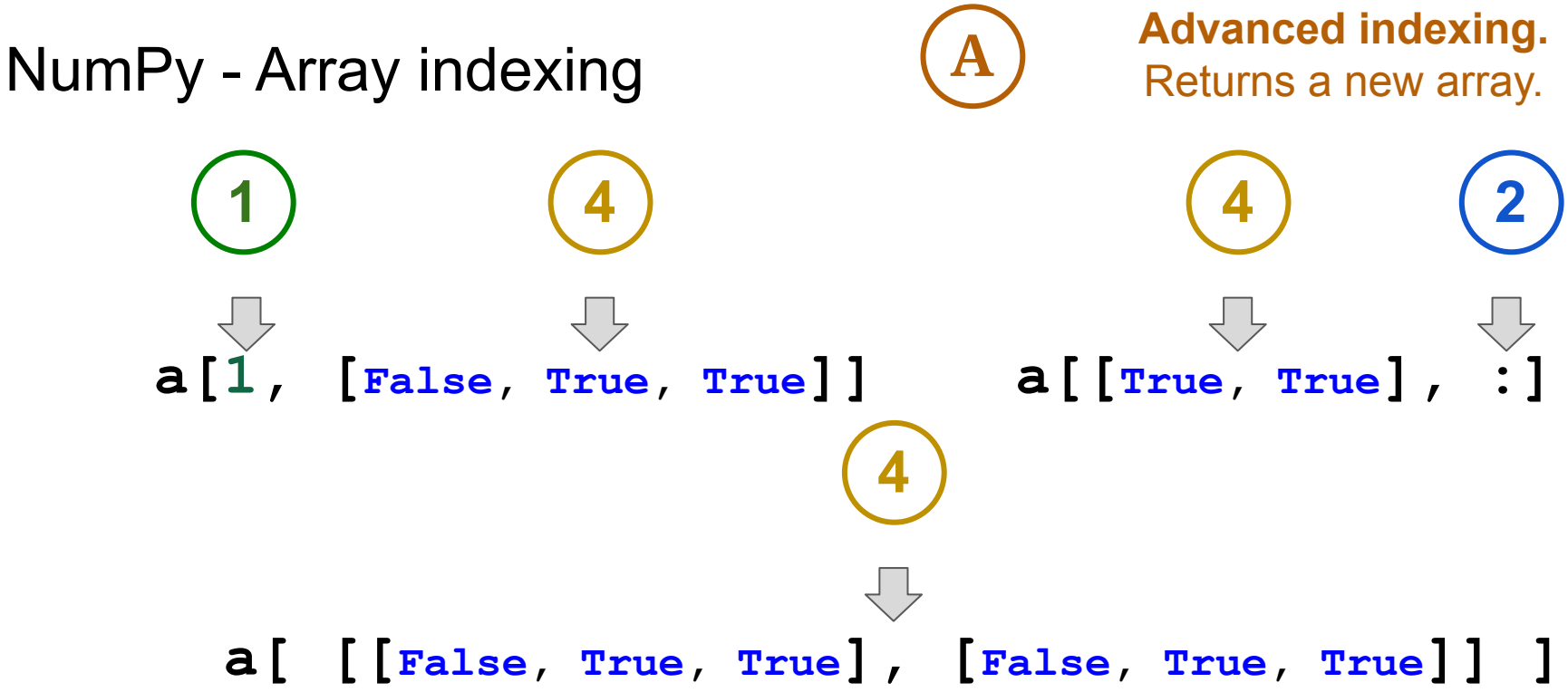
a[2, [1,-1]]



a[[1,3,0], -1::-2]



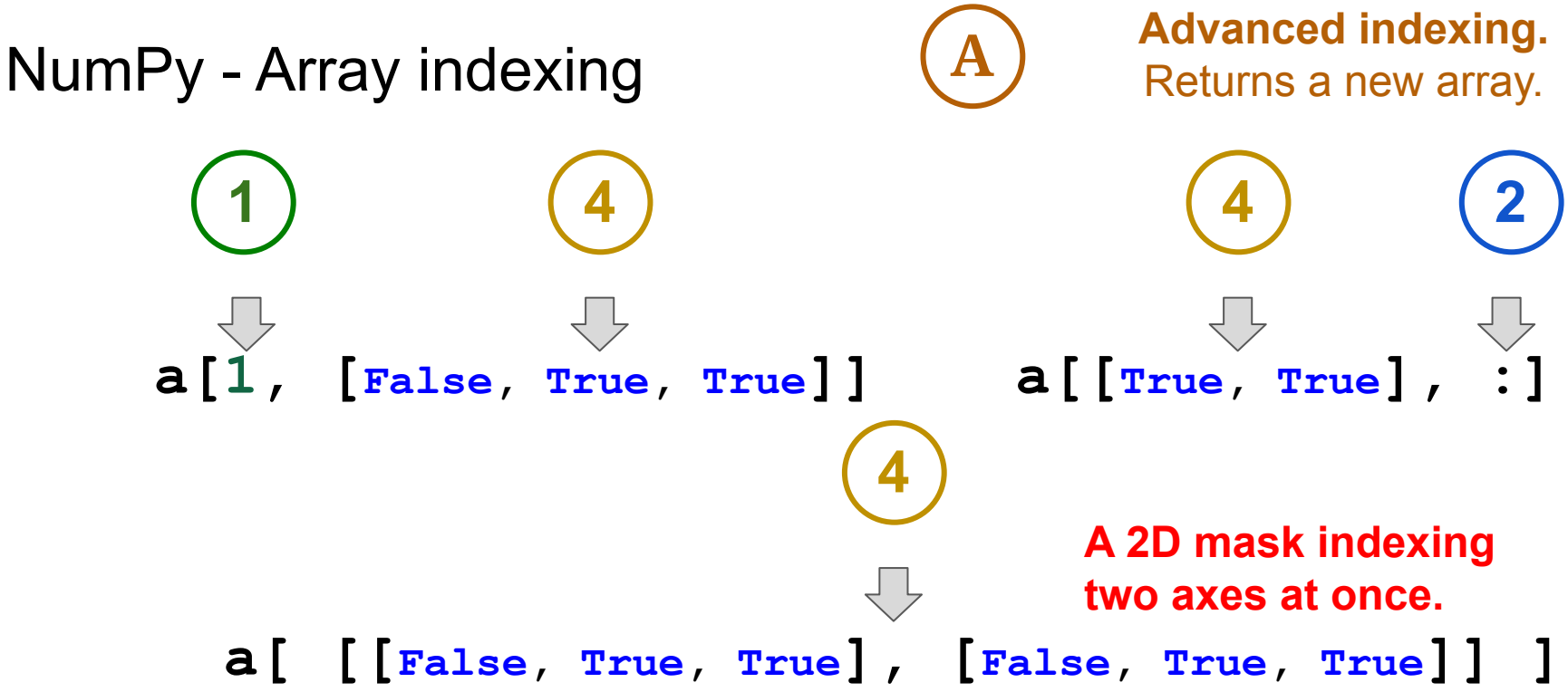
NumPy - Array indexing



Advanced indexing:

- Will happen if either integer sequence or mask indexing is used at least once.
- A **new array** is returned with the selected elements copied into it.

NumPy - Array indexing



Advanced indexing:

- Will happen if either integer sequence or mask indexing is used at least once.
- A **new array** is returned with the selected elements copied into it.

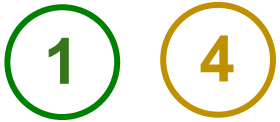
NumPy - Array indexing



Advanced indexing.
Returns a new array.

```
a = np.array([[0, 1, 2], [10, 11, 12]], dtype=np.int64)   # a.shape == (2, 3)
print(a)                    #   [[ 0  1  2]
                            #    [10 11 12]]
```

```
print( a[1, [False, True, True]] )          # [11 12]
print( a[1, [False, True, True]].shape )    # (2,)
print( type(a[1, [False, True, True]]) )    # <class 'numpy.ndarray'>
```




```
print( a[[True, True], :] )           # [[ 0  1  2]
                                      #  [10 11 12]]
print( a[[True, True], :].shape )     # (2, 3)
```




```
print( a[ [[False, True, True], [False, True, True]] ] )          # [ 1  2 11 12]
print( a[ [[False, True, True], [False, True, True]] ].shape )    # (4,)
```



NumPy - Array indexing



Advanced indexing.
Returns a new array.

```
a = np.array([[0, 1, 2], [10, 11, 12]], dtype=np.int64)   # a.shape == (2, 3)
print(a)                    #   [[ 0  1  2]
                            #    [10 11 12]]
m = a % 2 == 0
print(m)                    #   [[ True False  True]
                            #    [ True False  True]]

print( a[m] )               # [ 0  2 10 12]
print( a[m].shape )         # (4,)
```

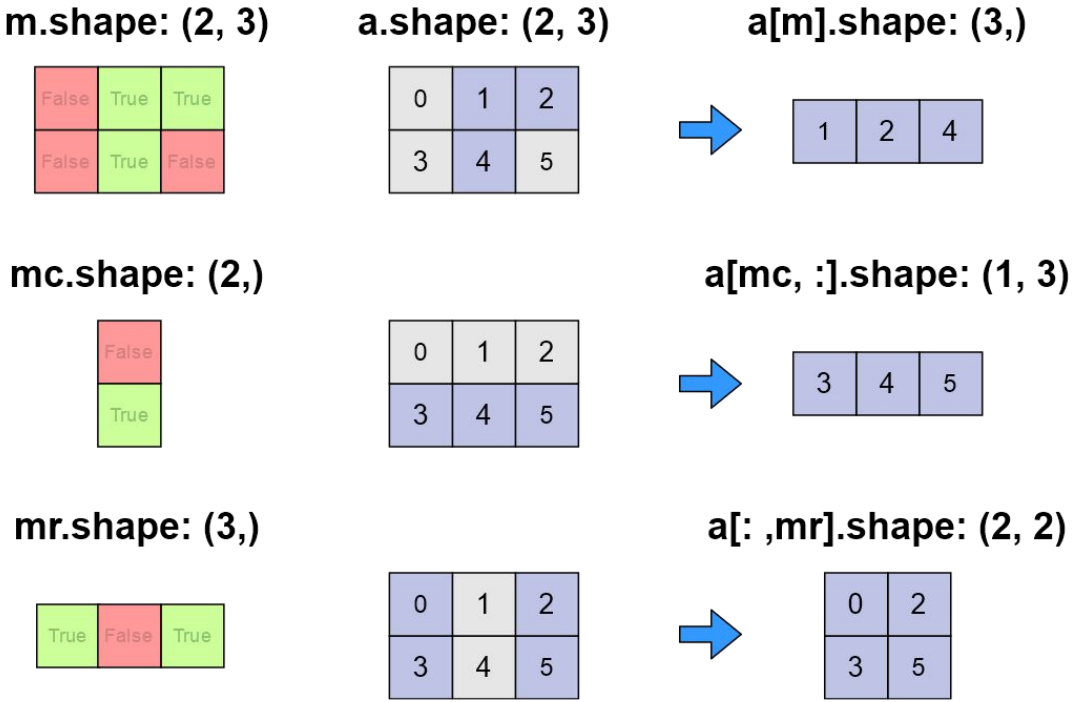


Masks are, of course, often coming from the result of another operation…

NumPy - Array indexing



Advanced indexing.
Returns a new array.



NumPy - Array indexing



Indexing an individual element of the nD array.
Returns a scalar.



Basic indexing.
Returns a view array over the original array.*



Advanced indexing.
Returns a new array.

Easy to remember: If an indexing of an array is not individual element indexing or basic indexing, then it is **advanced indexing.**